



Web Technologies

WET2VO

Teil 12: Microframeworks & Testing

Medientechnik und -design | SS 2026

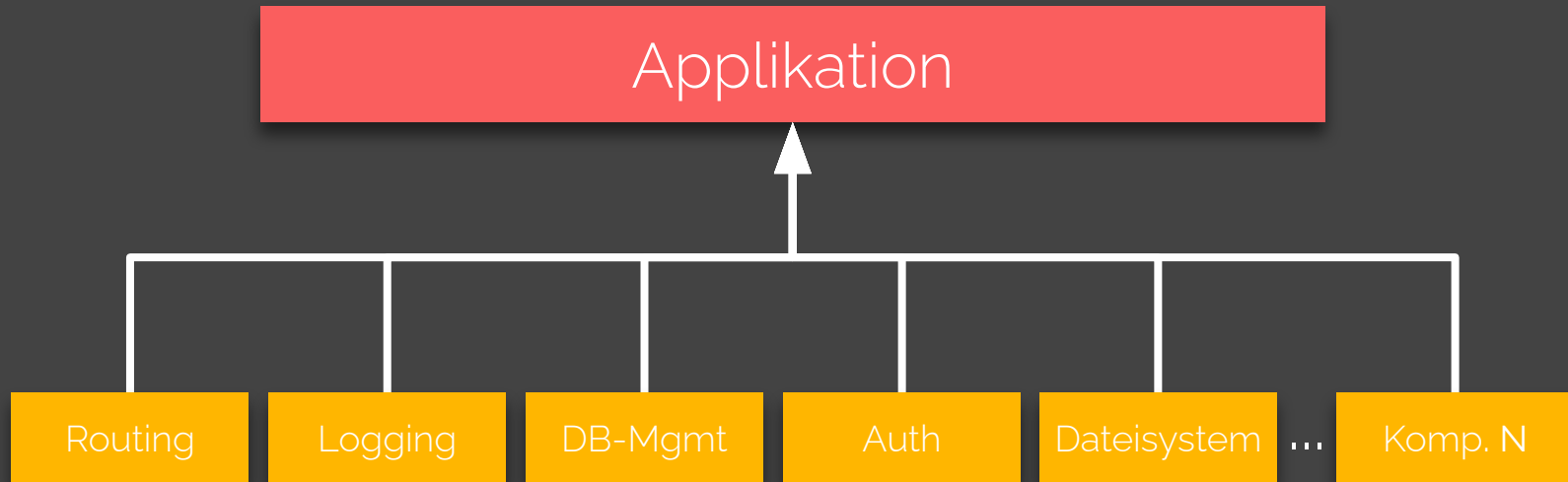
Microframeworks

Minimalistische(re) Frameworks fürs Backend

Full-Stack Frameworks



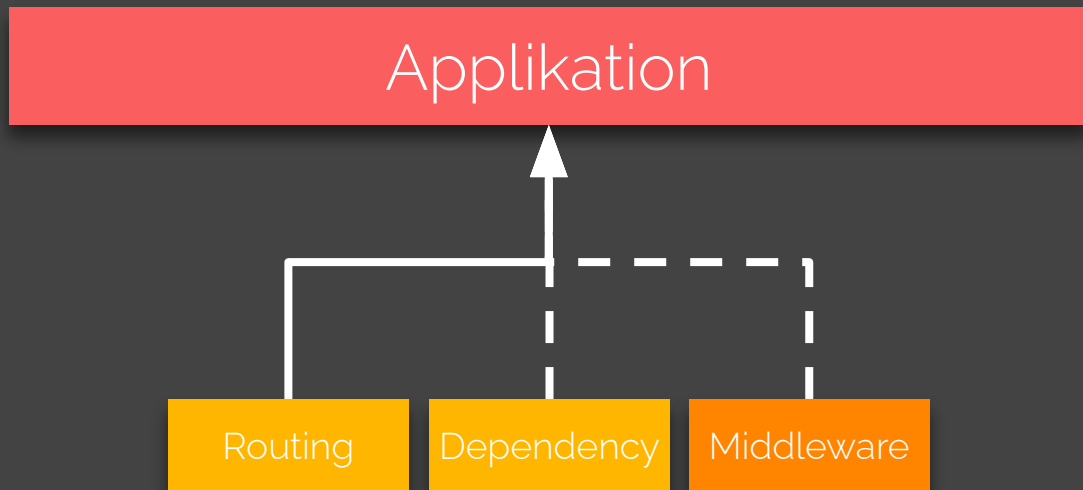
Symfony, Laravel, Laminas (Zend)





Micro- Frameworks

Slim, Mezzio, Flight, Fat-Free Framework



Slim Framework

A micro framework that helps you quickly write simple yet powerful web applications and APIs.

Slim Framework k



PHP Microframework

Version 4.15.2 (22. Mai 2026)

<https://www.slimframework.com/>



Slim Features

HTTP Router

Professioneller **Router** mit Callbacks für alle HTTP-Methoden.

Middleware

Middleware kann verwendet werden, um **Request und Response** zu verändern.

PSR-7 Unterstützung

Alle Komponenten, den PSR-7 **HTTP-Message-Standard** implementieren, werden unterstützt.

Dependency Injection

Dependencies (Komponenten) werden in einem **Container** (PSR-11) verwaltet und geladen.

Front Controller r



Alle Requests laufen an einer zentralen Stelle zusammen und werden dort behandelt (Design Pattern).

In PHP in einer Datei: `index.php`

URL-Rewriting am Server



.htaccess (braucht mod_rewrite und AllowOverride All):

```
<IfModule mod_rewrite.c>
```

```
RewriteEngine On
```

```
RewriteCond %{REQUEST_URI}::$1 ^(/.+)/(.*):~\2$
```

```
RewriteRule ^(.*) - [E=BASE:%1]
```

```
RewriteCond %{REQUEST_FILENAME} !-f
```

```
RewriteRule ^ index.php [QSA,L]
```

```
</IfModule>
```

Basis-Verzeichnis
ermitteln (falls
Applikation nicht im
Root des Webservers)

Wenn die Datei nicht
existiert, leite auf
index.php weiter

Hello World

Eine erste Applikation mit dem Slim-Framework.





Installation

```
$ composer require slim/slim
```

```
$ composer require slim/psr7
```

Verzeichnisstruktur:

```
helloworld/  
├─ public/  
│   ├── index.php  
│   └── .htaccess  
├─ vendor/  
├─ composer.json  
└─ composer.lock
```

Die Applikation



```
use Slim\Factory\AppFactory;
use Psr\Http\Message\ResponseInterface as Response;
use Psr\Http\Message\ServerRequestInterface as Request;

require __DIR__ . '/../vendor/autoload.php';

$app = AppFactory::create();

$app->setBasePath("/wet2vo-examples/vo12/slim-framework/helloworld/public");
$app->addErrorMiddleware(true, false, false);

$app->get("/", function (Request $request, Response $response, array $args) {
    $response->getBody()->write("Hello world!");
    return $response;
});

$app->run();
```

Full Example

Formularverarbeitung mit Templates, Logging, Dependency Injection, Error Handling und Middleware.





Installation

```
$ composer require slim/slim slim/psr7
```

Verzeichnisstruktur:

```
slimexample/  
├─ cache/  
├─ logs/  
├─ public/  
│   ├─ index.php  
│   └─ .htaccess  
├─ templates/  
├─ vendor/  
├─ composer.json  
└─ composer.lock
```

Initiale Schritte



```
use Slim\Factory\AppFactory;  
use Psr\Http\Message\ResponseInterface as Response;  
use Psr\Http\Message\ServerRequestInterface as Request;
```

```
require __DIR__ . "/../vendor/autoload.php";
```

```
$app = AppFactory::create();
```

```
$app->setBasePath("/wet2vo-examples/vo12/slim-framework/slimexample/public");
```

Dependencies (Logger, Templates etc.) in die App einbinden?

Dependency Injection (DI) Container verwenden!



DI-Container

```
$ composer require php-di/php-di
```

```
$containerBuilder = new ContainerBuilder();
```

```
$container = $containerBuilder→build();
```

```
AppFactory::setContainer($container);
```

```
$app = AppFactory::create();
```




DI-Container Prinzip

Dependency Injection Container (PHP-DI)

settings
array

myService1
MyService1

myService2
MyService2

myService3
MyService3

```
$containerBuilder→addDefinitions(  
    [  
        "settings" ⇒ [  
            "myService1" ⇒ "someValue",  
        ],  
    ],  
);
```

```
$containerBuilder→addDefinitions(  
    [  
        MyService1::class ⇒ function (ContainerInterface $c) {  
            $settings = $c→get("settings");  
            return new MyService1($settings["myService1"]);  
        },  
    ],  
);
```

Error-Logging Settings



Einstellungen in index.php definieren:

```
$containerBuilder→addDefinitions(  
    [  
        "settings" ⇒ [  
            "displayErrorDetails" ⇒ true,  
            "logErrors" ⇒ true,  
            "logErrorDetails" ⇒ true,  
        ],  
    ],  
);
```

Einstellungen später abfragen:

```
$errorSettings = $container→get("settings")["displayErrorDetails"];  
$errorSettings = $this→get("settings")["displayErrorDetails"]; // In Callbacks
```

Dependency: Monolog



```
$ composer require monolog/monolog
```

Settings hinterlegen:

```
$containerBuilder→addDefinitions(  
    [  
        "settings" ⇒ [  
            // ...  
            "logger" ⇒ [  
                "name" ⇒ "slimexample-logger",  
                "path" ⇒ "../logs/app.log",  
                "level" ⇒ Level::Debug,  
            ],  
        ],  
    ],  
);
```

Dependency im Container registrieren:

```
$containerBuilder→addDefinitions(  
    [  
        LoggerInterface::class ⇒  
            function (ContainerInterface $container) {  
                $settings = $container→get("settings");  
                $loggerSettings = $settings["logger"];  
                $logger = new Logger($loggerSettings["name"]);  
                $fileHandler = new StreamHandler(  
                    $loggerSettings["path"],  
                    $loggerSettings["level"]);  
                $logger→pushHandler($fileHandler);  
                return $logger;  
            },  
    ],  
);
```

Dependency: Monolog



Logging-Aufrufe absetzen:

```
$logger = $container→get(LoggerInterface::class);  
$logger→info("Logger dependency created.");
```

```
$logger→debug("Debug message");  
$logger→warning("Warning");  
$logger→error("Serious error");  
$logger→critical("Absolutely critical");
```

Output in logs/app.log:

```
[2026-06-23T06:50:00+00:00] slimexample-logger.INFO: Logger dependency created. [] []
```

Dependency: Latte



```
$ composer require latte/latte
```

Settings hinterlegen:

```
$containerBuilder→addDefinitions(  
    [  
        "settings" ⇒ [  
            // ...  
            "template" ⇒ [  
                "path" ⇒ "../templates",  
                "cache" ⇒ "../cache",  
                "auto_refresh" ⇒ true,  
            ],  
        ],  
    ],  
);
```

RouteParser im Container hinzufügen:

```
$container→set("routeParser", $app→getRouteCollector()→getRouteParser());
```

Dependency: Latte



```
$ composer require latte/latte
```

Dependency im Container registrieren:

```
$containerBuilder→addDefinitions(  
    [  
        "Engine::class" ⇒ function (ContainerInterface $c) {  
            $settings = $c→get("settings");  
            $templateSettings = $settings["template"];  
  
            $latte = new Engine();  
            $latte→setLoader(new FileLoader($templateSettings["path"]));  
            $latte→setCacheDirectory($templateSettings["cache"]);  
            $latte→setAutoRefresh($templateSettings["auto_refresh"]);  
  
            $routeParser = $container→get("routeParser");  
            $latte→addFunction("url_for", fn(string $name, array $data = [], array $query = []): string  
                ⇒ $routeParser→urlFor($name, $data, $query),  
            );  
            return $latte;  
        }]);
```

Dependency: Latte



Template rendern. Eine Klasse `TemplateRenderer` erzeugt ein PSR-7 Response-Objekt:

```
final class TemplateRenderer
{
    public function __construct(
        private Engine $engine,
    ) {}

    public function template(ResponseInterface $response,
                             string $template,
                             array $data = []): ResponseInterface
    {
        $output = $this->engine->renderToString($template, $data);
        $response->getBody()->write($output);

        return $response;
    }
}
```

Dependency: Latte



Aufruf zum Rendern eines Templates:

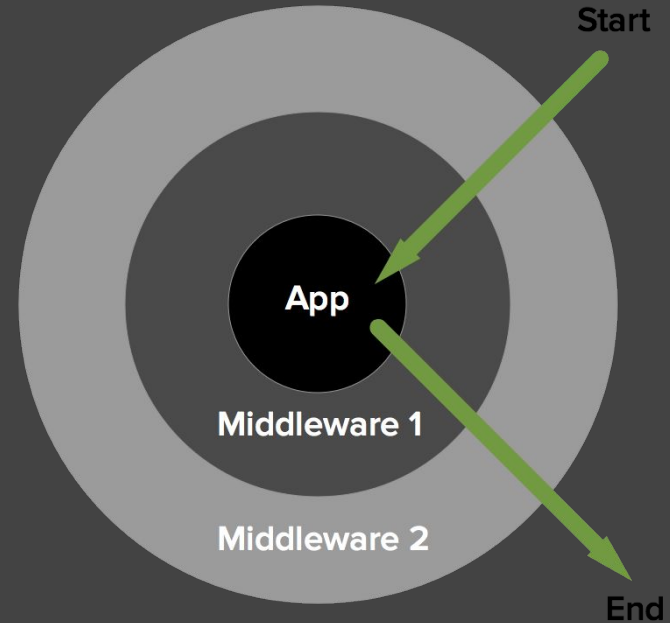
```
return $this->get(TemplateRenderer::class)->template($response,  
    "template.latte",  
    ["param1" => "value1"]  
);
```




Slim Middleware

Verändern von Request und Response vor und nach dem Ausführen der Applikationslogik.

Z.B. Caching oder Authentifikation





Middleware:

Error-Handler

Error-Handler registrieren:

```
$settings = $container->get("settings");  
$app->addErrorMiddleware(  
    $settings["displayErrorDetails"],  
    $settings["logErrors"],  
    $settings["logErrorDetails"],  
    $logger,  
);
```

Fängt Fehler und Exceptions ab, z.B. 404 Fehler



Routing

Bestandteile einer Route:

- Protokoll (GET, POST, PUT, DELETE, OPTIONS, PATCH)
- URI (URL)
- Callback-Methode (Verarbeitung des Requests, Generierung der Response)

Beispiel aus Hello World:

```
$app→get("/", function (Request $request, Response $response, array $args) {  
    $response→getBody()→write("Hello world!");  
    return $response;  
});
```

Aufruf einer Route



Beispiel aus Hello World:

```
$app→get("/", function (Request $request, Response $response, array $args) {  
    $response→getBody()→write("Hello World!");  
    return $response;  
});
```

Route, Verzeichnis und URL im Browser:

Route: /

Verzeichnis: /wet2vo-examples/vo12/slim-framework/slimexample/public/

URL: http://localhost:8080/wet2vo-examples/vo12/slim-framework/slimexamples/public/

Route: /list

Verzeichnis: /wet2vo-examples/vo12/slim-framework/slimexample/public/list

URL: http://localhost:8080/wet2vo-examples/vo12/slim-framework/slimexamples/public/list

Formular mit Slim



Formular anzeigen (GET-Route):

```
$app→get("/", function (Request $request, Response $response, array $args) {  
    return $this→get(TemplateRenderer::class)→template($response, "form.latte");  
});
```

Template form.latte (Ausschnitt):

```
{parameters string $placeholder = ''}  
<body>  
    <h1>Tell Me Your Name</h1>  
    <form action="{url_for('result')}}" method="post">  
        <label for="name">Name:</label>  
        <input type="text" id="name" name="name" value="{ $placeholder}">  
        <button type="submit">Send</button>  
    </form>  
</body>
```

Formular mit Slim



Ergebnis anzeigen (POST-Route):

```
$app→post("/", function (Request $request, Response $response, array $args) {  
    $data = $request→getParsedBody();  
    $name = $data["name"];  
    return $this→get(TemplateRenderer::class)→template($response,  
                                                         "result.latte",  
                                                         ["name" ⇒ $name]);  
})→setName("result");
```

Template result.latte (Ausschnitt):

```
<body>  
    <h1>Form Result</h1>  
    <p>The form was submitted by { $name }.</p>  
</body>
```

Routing- Platzhalter



Alternative GET-Route mit Platzhalter:

```
$app→get("/{placeholder}[/]",  
    function (Request $request, Response $response, array $args) {  
        return $this→get(TemplateRenderer::class)→template(  
            $response,  
            "form.latte",  
            ["placeholder" ⇒ $args["placeholder"]]  
        );  
    });
```



Slim-Skeleton

```
$ composer create-project slim/slim-skeleton path
```

Fertiges Slim-Starterprojekt mit aufgeteilter Struktur:

```
slim-skeleton/  
├─ app/  
│   ├── dependencies.php  
│   ├── middleware.php  
│   ├── repositories.php  
│   ├── routes.php  
│   └── settings.php  
├─ logs/  
├─ public/  
│   ├── index.php  
│   └── .htaccess  
├─ src/ (Klassen zur Verarbeitung der Routen nach dem Action-Domain-Responder Modell)  
└─ tests/
```


Testing

Unit-Tests für PHP-Applikationen



Testen in a Nutshell

Wann?

Hauptsächlich **vor und während** der Entwicklung.

Nach der Entwicklung ist es eigentlich zu spät.

Was?

Öffentliche **Klassen**, **Funktionen** und **Methoden** einer Applikation.

Wie?

Unit-Tests, funktionale Tests, Test-Driven-Development (TDD), Behavior-Driven-Development (BDD).

PHPUnit

PHPUnit

PHP Unit-Testing Framework

Version 13.2.1 (15. Juni 2026)

<https://phpunit.de/>

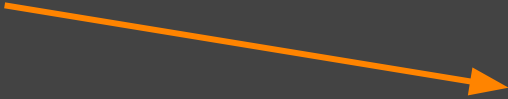


PHPUnit

```
$ composer require --dev phpunit/phpunit
```

Konfiguration (phpunit.xml):

```
<?xml version="1.0" encoding="UTF-8"?>
<phpunit xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
          xsi:noNamespaceSchemaLocation="https://schema.phpunit.de/13.2/phpunit.xsd"
          bootstrap="tests/bootstrap.php">
  <testsuites>
    <testsuite name="favoriteSubject">
      <directory>tests</directory>
    </testsuite>
  </testsuites>
  <source>
    <include>
      <directory>src</directory>
    </include>
  </source>
</phpunit>
```



```
require dirname(__DIR__) . "/vendor/autoload.php";
```

Code-Klasse



```
class FavoriteMTDSubject
{
    private(set) string $favoriteSubject;

    public function __construct(string $favoriteSubject) {
        if (mb_trim($favoriteSubject) === "") {
            throw new InvalidArgumentException("Favorite subject cannot be empty.
                                           You must choose!");
        }
        $this->favoriteSubject = $favoriteSubject;
    }

    // weitere Methoden
}
```

Code-Klasse



```
class FavoriteMTDSubject
{
    // Konstruktor

    public function say(): string {
        return "The best subject in MTD is $this->favoriteSubject!";
    }

    public function respondTo(string $input): string {
        $input = mb_strtolower($input);
        $myFavoriteSubject = mb_strtolower($this->favoriteSubject);
        if (mb_strpos($input, $myFavoriteSubject) === false) {
            throw new Exception("Never! $this->favoriteSubject is the best subject
                                in MTD!");
        }
        return "Absolutely true!";
    }
}
```

Test-Klasse: **Test 1**



```
class FavoriteMTDSubjectTest extends TestCase
{
    public function testConstructorThrowsExceptionOnEmptyFavoriteSubject()
    {
        $this->expectException(InvalidArgumentException::class);
        new FavoriteMTDSubject("");
    }

    // ...
}
```

Test-Klasse: Test 2



```
class FavoriteMTDSubjectTest extends TestCase
{
    // ...
    public function testSayFavoriteSubject()
    {
        $subject = new FavoriteMTDSubject("Web Technologies");
        $this->assertEquals(
            "The best subject in MTD is Web Technologies!",
            $subject->say()
        );
    }
    // ...
}
```


Test-Klasse: Test 3



```
class FavoriteMTDSubjectTest extends TestCase
{
    // ...
    public function testRespondToInAgreement()
    {
        $subject = new FavoriteMTDSubject("Web Technologies");
        $opinion = "Web Technologies is the best!";

        $this->assertEquals("Absolutely true!", $subject->respondTo($opinion));
    }
    // ...
}
```

Test-Klasse: Test 3



```
class FavoriteMTDSubjectTest extends TestCase
{
    // ...
    public function testRespondToInDisagreement()
    {
        $subject = new FavoriteMTDSubject("Web Technologies");
        $opinion = "I love Algorithmic Thinking!";

        $this->expectException(Exception::class);

        $subject->respondTo($opinion);
    }
}
```

Tests aufrufen



Auf der Commandline:

```
$ vendor/bin/phpunit -c phpunit.xml
```

```
PHPUnit 13.2.1 by Sebastian Bergmann and contributors.
```

```
Runtime: PHP 8.5.7
```

```
Configuration: phpunit.xml
```

```
....
```

```
4 / 4 (100%)
```

```
Time: 00:00.025, Memory: 18.00 MB
```

```
OK (4 tests, 4 assertions)
```

Code-Coverage Report:

```
$ vendor/bin/phpunit -c phpunit.xml --coverage-html coverage
```

Funktionale Tests



```
<?php

namespace SlimExampleTest;

use PHPUnit\Framework\TestCase;
use Slim\Factory\AppFactory;
use Slim\Psr7\Factory\ServerRequestFactory;

final class HomePageTest extends TestCase
{
    public function testHomePageShowsForm(): void
    {
        $app = AppFactory::create();
        // Routen und Middleware wie in der Applikation registrieren ...

        $request = (new ServerRequestFactory())->createServerRequest("GET", "/");
        $response = $app->handle($request);

        $this->assertSame(200, $response->getStatusCode());
        $this->assertStringContainsString("Tell Me Your Name", (string) $response->getBody());
    }
}
```



pestphp/pest

```
$ composer require pestphp/pest  
  --dev --with-all-dependencies  
$ ./vendor/bin/pest --init  
$ ./vendor/bin/pest
```

```
test('example', function () {  
    expect(true)→toBeTrue();  
});
```

<https://pestphp.com/>

Beispiel mit Pest



Test 1 (Konstruktor):

```
test("constructor rejects an empty subject", function () {  
    new FavoriteMTDSubject("");  
})→throws(InvalidArgumentException::class);
```

Test 2 (say()):

```
test("say returns the favorite subject", function () {  
    subject = new FavoriteMTDSubject("Web Technologies");  
    expect($subject→say())→toBe("The best subject in MTD is Web Technologies!");  
});
```



Klausur & Vorbereitung

Klausur-Hangout



Klausur

- e-Klausur in Moodle am 29.06.2026.
 - 13:00 & 14:40 Uhr im WHS.
- Stoff als PDF im eLearning-Kurs.
- WET2 Klausur-Hangout? Bitte um Termin!
 - z.B. Freitag, 26.06.2026?
 - MS-Teams-Call zur Beantwortung offener Fragen.

Credits



- S. 6: Slim Framework Logo: <https://www.slimframework.com/>
- S. 35: PHPUnit Logo: <https://phpunit.de/>
- S. 45: PEST Logo: <https://pestphp.com/>